

Chapter 7 — Column Definitions

(columns/itemColumns.tsx)

File: `src/features/react-crud-v4/columns/itemColumns.tsx` **One job:** Declarative config for the table. To add, remove, or reorder columns — edit only this file.

7.1 Why a Separate Columns File?

Column definitions are a mix of configuration (which fields to show, how wide) and rendering (how to display each cell). If they live inside `ItemTable.tsx`, the table component becomes 500+ lines and hard to navigate.

Extracting them to `itemColumns.tsx` means:

- Adding a column = edit one file, no understanding of the table component needed
- The table component stays focused on rendering logic
- Column definitions can be tested independently

The rule: To add/remove/reorder columns, edit only `itemColumns.tsx`. No other file should change.

7.2 FILTERABLE_COLUMNS — Line by Line

```
export const FILTERABLE_COLUMNS: FilterableColumn[] = [  
  { id: "name",          label: "Name",          type: "string",  
    valueType: "text" },  
  { id: "year",          label: "Year",          type: "string",  
    valueType: "dropdown" },  
  { id: "currentTermStatus", label: "Current Status", type: "string",  
    valueType: "dropdown" },  
  { id: "previousTermStatus", label: "Previous Status", type: "string",  
    valueType: "dropdown" },  
  { id: "daysAbsentYtd",   label: "Days Absent (YTD)", type: "number" },  
  { id: "lastActivityDate", label: "Last Activity Date", type: "date" },  
];
```

What is `FILTERABLE_COLUMNS`? The `TableFilterBuilder` component uses this array to know which columns users can filter on, what type they are, and how to render the filter input.

id — must match the `accessorKey` of the corresponding column definition. This is how `TableFilterBuilder` knows which column to apply the filter to.

type: "string" | "number" | "date" — determines which operators are available:

- "string" → operators: equals, contains, not equals
- "number" → operators: equals, greater than, less than, $\geq$, $\leq$
- "date" → operators: before, after

valueType: "text" | "dropdown" — for string columns only:

- "text" → shows a text input (user types a value)
- "dropdown" → shows a dropdown populated from the actual row values in the loaded data

Why export it? Both `itemColumns.tsx` (to define it) and `ItemTable.tsx` (to pass it to `TableFilterBuilder` and register the custom `filterFn`) need it. Exporting from one place prevents duplication.

7.3 DEFAULT_COLUMN_VISIBILITY — Line by Line

```
export const DEFAULT_COLUMN_VISIBILITY: VisibilityState = {
  homeClass: false,
  coreClass: false,
  house: false,
  homeRoom: false,
  homeTeacher: false,
  dean: false,
  deputy: false,
};
```

VisibilityState — TanStack Table's type for column visibility. It's `Record<string, boolean>` — column ID → visible or not.

Why hide columns by default? The table has 24 columns. Showing all 24 by default would be overwhelming and require horizontal scrolling. We hide columns that are useful for filtering/export but clutter the default view. Users can show them via the "Columns" dropdown toggle.

Columns not listed here are visible by default. You only need to list columns you want hidden. To show a hidden column by default, remove it from this object.

VisibilityState is passed to TanStack Table's initialState:

```
initialState: {
  columnVisibility: DEFAULT_COLUMN_VISIBILITY,
}
```

This sets the starting visibility without making it a controlled state — the user can toggle columns and the table remembers their choice locally.

7.4 buildItemColumns — Line by Line

```
type ColumnConfig = {
  actions?: ActionsConfig<ItemRow>;
};
```

```
export const buildItemColumns = ({ actions }: ColumnConfig = {}):
ColumnDef<ItemRow>[] => [
```

ColumnConfig — a local type for the function's parameter. Only one option for now (**actions**), but using an object parameter makes it easy to add more options later without breaking callers.

{ actions }: ColumnConfig = {} — destructuring with a default value. If called with no arguments (**buildItemColumns()**), **actions** is **undefined**. This means the actions column is omitted automatically.

ColumnDef<ItemRow>[] — an array of TanStack Table column definitions, typed to **ItemRow**. TypeScript checks that **accessorKey** values are valid keys of **ItemRow**.

The Select Column

```
{
  id: "select",
  header: ({ table }) => (
    <Checkbox
      checked={table.getIsAllPageRowsSelected()}
      onCheckedChange={() => table.toggleAllPageRowsSelected(!checked)}
      aria-label="Select all rows on this page"
    />
  ),
  cell: ({ row }) => (
    <Checkbox
      checked={row.getIsSelected()}
      onCheckedChange={() => row.toggleSelected(!checked)}
      aria-label="Select row"
    />
  ),
  enableSorting: false,
  enableGlobalFilter: false,
  size: 1,
},
```

id: "select" — this column has no **accessorKey** because it doesn't display data from the row. It renders a checkbox. Columns without **accessorKey** must have an **id**.

header: ({ table }) => ... — the header cell receives the **table** instance. We use it to:

- **table.getIsAllPageRowsSelected()** — returns **true** if every row on the current page is checked. Used to show the "select all" checkbox as checked.
- **table.toggleAllPageRowsSelected(!checked)** — checks or unchecks all rows on the current page.

cell: ({ row }) => ... — each data cell receives the **row** instance:

- **row.getIsSelected()** — returns **true** if this row is checked

- `row.toggleSelected(!checked)` — checks or unchecks this row

!!checked — Shadcn's `onCheckedChange` can return `boolean | "indeterminate"`. `!!` converts any value to a boolean: `true → true`, `false → false`, `"indeterminate" → true`. This normalizes the value for TanStack Table which expects a boolean.

aria-label="Select all rows on this page" — WCAG accessibility requirement. Screen readers announce this text when the user focuses the checkbox. Without it, the checkbox is announced as just "checkbox" with no context — the user doesn't know what it does.

enableSorting: false — the select column cannot be sorted (clicking the header doesn't sort). Makes sense — there's no meaningful sort order for checkboxes.

enableGlobalFilter: false — the select column is excluded from global search. Searching "true" should not match selected rows.

size: 1 — the smallest possible column. The table normalizes all column sizes to percentages. `1` out of a total of `~100` = `~1%` width.

A Data Column (Name)

```
{
  accessorKey: "name",
  header: ({ column }) => <TableHeaderButton label="Name" column={column} />,
  cell: ({ row }) => {
    const name = row.original.name ?? "";
    if (name.length <= MAX_NAME_CHARS) return name;
    return (
      <Tooltip>
        <TooltipTrigger>{name.slice(0, MAX_NAME_CHARS)}...</TooltipTrigger>
        <TooltipContent>{name}</TooltipContent>
      </Tooltip>
    );
  },
  size: 14,
},
```

accessorKey: "name" — tells TanStack Table to read `row.original.name` for this column's value.

TypeScript checks that `"name"` is a valid key of `ItemRow`. If you rename the field in `ItemRow`, TypeScript shows an error here.

header: ({ column }) => <TableHeaderButton label="Name" column={column} /> — the header cell receives the `column` instance. `TableHeaderButton` is a shared component that renders a sortable column header button. Clicking it toggles sort ascending/descending. The `column` instance is passed so `TableHeaderButton` can call `column.toggleSorting()`.

cell: ({ row }) => { ... } — the data cell renderer. `row.original` is the raw `ItemRow` object.

Truncation with tooltip:

```
const MAX_NAME_CHARS = 40;
if (name.length <= MAX_NAME_CHARS) return name;
return (
  <Tooltip>
    <TooltipTrigger>{name.slice(0, MAX_NAME_CHARS)}...</TooltipTrigger>
    <TooltipContent>{name}</TooltipContent>
  </Tooltip>
);
```

Long names are truncated to 40 characters with an ellipsis. Hovering shows the full name in a tooltip. This prevents long names from breaking the table layout.

A Simple Data Column (Year)

```
{
  accessorKey: "year",
  header: ({ column }) => <TableHeaderButton label="Year" column={column} />,
  cell: ({ row }) => <span className="text-muted-foreground">
    {row.getValue("year")}</span>,
  enableGlobalFilter: false,
  filterFn: (row, columnId, value) => String(row.getValue(columnId)).trim() ===
    String(value).trim(),
  size: 4,
},
```

cell: ({ row }) => ... — wraps the value in a span with muted color. Secondary data (year, class, etc.) is displayed in a lighter color to visually de-emphasize it compared to the primary data (name).

row.getValue("year") — gets the value for the "year" column from TanStack Table's internal state. This is equivalent to **row.original.year** but goes through TanStack Table's value pipeline (which can apply transformations).

enableGlobalFilter: false — excludes this column from global search. Searching "10" should not match all Year 10 students — it would return too many results. Year filtering is done via the server-side year dropdown.

filterFn: (row, columnId, value) => String(row.getValue(columnId)).trim() === String(value).trim() — a custom filter function for this column. When **TableFilterBuilder** applies a filter to the year column, it uses this function instead of the default. The **.trim()** calls handle whitespace in the data.

A Badge Column (Status)

```
{
  accessorKey: "currentTermStatusId",
  header: ({ column }) => <TableHeaderButton label="Current Term" column={column} />,
  cell: ({ row }) => (
    <ContrastBadge
      label={row.original.currentTermStatus}
      colorHex={row.original.currentTermStatusColor}
    />
  ),
  enableGlobalFilter: false,
  size: 8,
},
```

accessorKey: "currentTermStatusId" — the column is keyed on the numeric ID (for sorting — numbers sort correctly). But the cell renders the status name and color badge using `row.original.currentTermStatus` and `row.original.currentTermStatusColor`.

<ContrastBadge label={...} colorHex={...} /> — a shared component that renders a colored badge. It automatically calculates whether to use white or black text based on the background color for WCAG contrast compliance.

This is a good example of why `ItemRow` has both `currentTermStatusId` (for sorting) and `currentTermStatus` + `currentTermStatusColor` (for display). The column definition uses all three.

The Actions Column (Conditional)

```
...(actions
  ? [
    {
      id: "actions",
      header: () => null,
      cell: ({ row }: { row: { original: ItemRow } }) =>
        renderActionsCell(row.original, actions),
      enableSorting: false,
      enableGlobalFilter: false,
      size: actions.quick && actions.quick.length > 0 ? 1 + actions.quick.length
    : 1,
    } satisfies ColumnDef<ItemRow>,
  ]
  : []),
```

...(actions ? [...] : []) — conditional spread. If `actions` is provided, spread an array containing the actions column. If not, spread an empty array (adds nothing). This is how you conditionally include a column without an `if` statement.

header: () => null — the actions column has no header text. Returning `null` renders nothing.

`cell: ({ row }) => renderActionsCell(row.original, actions)` — `renderActionsCell` is a shared utility from `TableActions.tsx`. It reads the `actions` config and renders either quick action buttons or a ":" dropdown menu (or both).

`size: actions.quick && actions.quick.length > 0 ? 1 + actions.quick.length : 1` — dynamic size based on how many quick action buttons there are. Each quick action button needs ~1 size unit. If there are 2 quick actions, the column gets size 3 (1 base + 2 buttons).

`} satisfies ColumnDef<ItemRow>` — the `satisfies` operator checks that the object matches the type WITHOUT widening it.

Compare with `as`:

```
// "as" – forces the type, hides errors
{ id: "actions", typo: "oops" } as ColumnDef<ItemRow> // no error – "as"
suppresses it

// "satisfies" – checks the type, catches errors
{ id: "actions", typo: "oops" } satisfies ColumnDef<ItemRow> // ERROR – "typo" is
not in ColumnDef
```

`satisfies` is the safer choice when you want TypeScript to verify the object matches a type.

7.5 The `ActionsConfig` Pattern

```
export type ActionItem<T> = {
  icon: LucideIcon;
  label: string;
  onClick: (row: T) => void;
  variant?: "default" | "destructive";
};

export type ActionsConfig<T> = {
  quick?: ActionItem<T>[];
  menu?: (ActionItem<T> | "separator")[];
};
```

Why a declarative config instead of hardcoded buttons?

The column file should not contain business logic (what happens when you click Edit). Instead, it accepts a config object describing the actions and renders them generically. The actual handlers come from the page hook — testable in isolation.

quick — icon buttons shown directly in the row (always visible). Good for 1–2 primary actions. Rendered as icon-only buttons for space efficiency.

menu — items in a ":" dropdown menu. Good for 3+ actions or secondary actions. Rendered as a `DropDownMenu`.

"separator" — a visual divider line in the dropdown menu between groups of actions. The string literal "separator" is used as a discriminant — `renderActionsCell` checks `item === "separator"` to render a `<DropDownMenuSeparator>`.

variant: "destructive" — renders the button/menu item in red, signaling a dangerous action. Used for delete actions.

How it's used in `ItemTable.tsx`:

```
const actions = useMemo<ActionsConfig<ItemRow>>() => ({
  quick: [
    { icon: EditIcon, label: "Edit", onClick: (row) => onEdit(row.id) },
  ],
  menu: [
    { icon: EditIcon, label: "Edit", onClick: (row) => onEdit(row.id) },
    "separator",
    { icon: Trash2Icon, label: "Delete", variant: "destructive", onClick: (row)
=> onDeleteSelected([row.id]) },
  ],
}),
[onEdit, onDeleteSelected],
);
```

The `actions` object is wrapped in `useMemo` because it's passed to `buildItemColumns`, which is also memoized. Without `useMemo`, a new `actions` object is created on every render, causing `columns` to rebuild, causing TanStack Table to re-initialize.

7.6 Column Width Normalization

```
// In ItemTable.tsx
const totalSize = table.getVisibleFlatColumns().reduce((sum, col) => sum +
col.getSize(), 0);
const getColWidth = (size: number) => `${((size / totalSize) * 100).toFixed(2)}%`;
```

TanStack Table's `size` values are proportions, not percentages. The `name` column has `size: 14`. The `year` column has `size: 4`. These are relative to each other.

`getColWidth` normalizes them to percentages based on the currently visible columns. If the user hides the `homeClass` column (size 6), the remaining columns expand proportionally to fill the space.

Why `table-fixed` CSS? Without `table-fixed`, the browser calculates column widths based on content. This causes layout shifts as data loads (columns jump around). With `table-fixed`, widths are fixed by the `style={{ width: getColWidth(header.getSize()) }}` attribute — stable layout regardless of content.

7.7 Common Mistakes

✗ Using `accessorKey` that doesn't match `ItemRow`

```
// WRONG – TypeScript error: "studentname" is not a key of ItemRow
{ accessorKey: "studentname", ... }

// CORRECT – matches the exact key in ItemRow
{ accessorKey: "name", ... }
```

✗ Forgetting `enableGlobalFilter: false` on ID/date columns

```
// WRONG – searching "2024" matches all rows with that year in any column
{ accessorKey: "lastActivityDate", ... }

// CORRECT – date columns excluded from global search
{ accessorKey: "lastActivityDate", enableGlobalFilter: false, ... }
```

✗ Not wrapping `buildItemColumns` in `useMemo`

```
// WRONG – new columns array every render → TanStack Table re-initializes
const columns = buildItemColumns({ actions });

// CORRECT – stable reference
const columns = useMemo(() => buildItemColumns({ actions }), [actions]);
```

✗ Forgetting to add new columns to `DEFAULT_COLUMN_VISIBILITY` when needed

```
// If you add a column that should be hidden by default, add it here:
export const DEFAULT_COLUMN_VISIBILITY: VisibilityState = {
  // ... existing
  newHiddenColumn: false, // add this
};
```

Next: [Chapter 8 — Page Hook](#)